VELoop Rewards — CAPTCHA Earn Module

1. Task Overview

Build the **CAPTCHA Earn Module** for VELoop Rewards using React.

This is a **frontend-only task** with a very strong focus on **UI/UX, interaction, animation and visual quality**.

The CAPTCHA itself is intentionally simple. The challenge is to transform a normally boring CAPTCHA activity into a:

- Premium experience
- Interactive experience
- Engaging experience
- Rewarding experience
- Trustworthy fintech-style experience
- Highly responsive experience

The final UI should **not feel static or generic**.

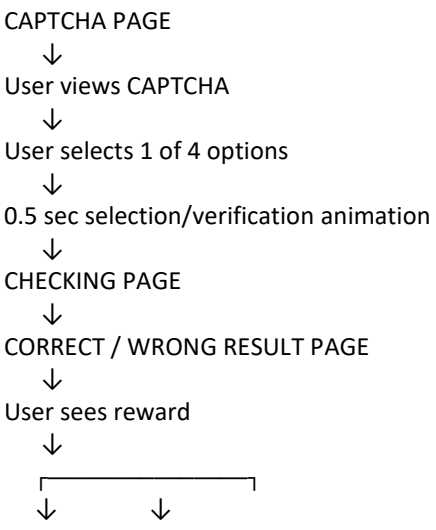
When a user opens the page, the experience should immediately feel like a professionally designed VELoop Rewards feature.

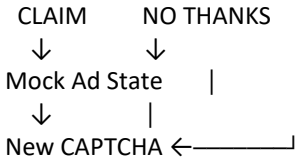
The primary evaluation criteria is UI/UX quality, not backend functionality.

---

2. Core User Flow

The complete frontend flow should be:





The previous CAPTCHA must **never be reused** after completing a challenge.

---

3. CAPTCHA Page

Create the primary CAPTCHA earning interface.

The page should display:

- CAPTCHA challenge
- Four answer options
- Reward information
- Supporting UI elements
- Clear interaction states
- Premium visual hierarchy

The page must immediately communicate that the user can **earn Gems by completing this activity**.

---

4. CAPTCHA Format

The CAPTCHA consists of a combination of:

- Letters
- Digits

Example:

A7K2P9

The CAPTCHA should be presented prominently and clearly.

The frontend may use mock/generated CAPTCHA data.

A real backend CAPTCHA/security system is **not required**.

5. Four Answer Options

Display exactly **4 selectable options**.

For every challenge:

- **1 option must be completely different**
- **3 options must look reasonably similar to the correct CAPTCHA**
- Only **1 of the 3 similar-looking options is actually correct**

Example:

Correct CAPTCHA

A7K2P9

Possible options

A7K2P9 ← Correct

AJK29P ← Similar but incorrect

A7L9P2 ← Similar but incorrect

X4M8Q1 ← Completely different

The examples above are only for understanding the logic.

The intern should create different combinations dynamically/mock from predefined data.

---

6. Important CAPTCHA Design Rule

The three similar options should be difficult enough to make the user actually inspect the CAPTCHA.

Do not make the incorrect options obviously wrong.

For example, avoid:

A7K2P9
A7K2P8
A7K2P7
123456

if the correct answer becomes too easy to identify.

Instead, use reordered or substituted characters/digits similar to the correct answer.

Example:

A7K2P9
AJK29P
A7L9P2
X4M8Q1

The goal is to create a **visual observation challenge**.

---

7. Answer Options UI

Do NOT use ordinary radio buttons or generic form controls.

Each option should feel like an interactive element.

The option should respond to:

- Hover
- Mouse movement
- Click
- Touch
- Selection

Possible interaction details:

- Smooth border transition
- Subtle elevation
- Glow
- Background transition
- Small scale response
- Active/pressed state
- Selected state
- Micro animation

The interaction should feel **fast and responsive**.

Avoid excessive animations.

---

8. Selection Behavior

There is **no Submit button**.

Once the user selects an option:

Option Selected
↓
0.5 second verification animation
↓
Checking Page

The selected answer should automatically be processed.

The user should not be able to select another answer after selecting one.

---

9. Selection Animation

After the user selects an answer, show a short interaction/verification animation.

Duration

Approximately 0.5 seconds.

The animation should feel:

- Smooth
- Fast
- Premium
- Responsive

Examples:

- Selected option briefly highlights
- Verification icon animation
- Subtle scanning effect
- Progress indicator
- Animated transition

Do not make the user wait unnecessarily.

The animation should communicate:

“Your answer is being verified.”

---

10. Checking Page

After the 0.5-second selection animation, transition to a dedicated **Checking/Verification state**.

The checking state should not look like a browser loading screen.

It should be part of the VELoop Rewards experience.

Possible content:

Checking your answer

with a subtle animated indicator.

The UI can use:

- Animated verification indicator
- Progress animation
- Scanning effect
- Rotating/spring element
- Subtle motion

Keep the duration short and natural.

Do not create an unnecessarily long loading state.

---

11. Correct Answer

If the selected answer is correct, show a dedicated result page.

Example:

Correct

CAPTCHA verified successfully

+1 Gem

The reward should be visually prominent.

The result should feel rewarding.

Possible elements:

- Animated success icon
- Gem reveal animation

- Reward counter animation
- Subtle particles
- Small celebration animation
- Smooth page transition

However:

Do NOT make it look like a casino, gambling app or children's game.

The reward experience should remain **premium and fintech-oriented**.

---

12. Wrong Answer

If the selected answer is incorrect, show a dedicated result page.

Example:

Incorrect

That answer wasn't correct

+0.5 Gems

The UI should not aggressively punish the user.

The experience should remain encouraging and professional.

The user should clearly understand:

- Their answer was incorrect
- They still received 0.5 Gems
- They can continue with another CAPTCHA

Use appropriate motion and visual feedback.

---

13. Reward Rules

The frontend should represent the following reward logic:

Correct Answer

+1 Gem

Wrong Answer

+0.5 Gems

No other reward values are required for this task.

The actual backend reward processing is **NOT part of this task**.

Use frontend/mock state where required.

---

14. Result Page Buttons

Both the correct and incorrect result pages must contain:

Primary Button

Claim

Secondary Button

No Thanks

The buttons must have a clear visual hierarchy.

The Claim button should visually communicate that it is the primary action.

The No Thanks button should be secondary but still clearly visible.

---

15. Claim Button — IMPORTANT

Do NOT implement actual Claim functionality.

The Claim functionality will be integrated later.

For this task, the intern only needs to build the **frontend UI/state placeholder** for the Claim interaction.

The future flow will eventually be:

Claim
↓
Rewarded Video Ad
↓

New CAPTCHA

But the actual:

- Advertisement
- Ad SDK
- Ad network
- Reward validation
- Reward transaction
- Backend API

must NOT be implemented.

---

16. Mock Ad State

Create a simple frontend placeholder for the future rewarded advertisement flow.

For example:

Preparing reward...

or an appropriate premium ad/loading state.

The intern should create **one mock ad/transition state** so that the future integration point is visually prepared.

Do not use a fake third-party advertisement.

Do not integrate a real ad network.

---

17. No Thanks Flow

When the user selects:

No Thanks

the user must be redirected to the CAPTCHA page.

A **completely new CAPTCHA** must be generated/displayed.

Example:

CAPTCHA #1

↓
Answer
↓
Result
↓
No Thanks
↓
CAPTCHA #2

The previous CAPTCHA must never appear again.

---

18. Claim Flow Placeholder

For the frontend demonstration, Claim should eventually lead back to the CAPTCHA experience through the mock ad state.

Example:

Result
↓
Claim
↓
Mock Rewarded Ad State
↓
New CAPTCHA

The new CAPTCHA must be different from the previous CAPTCHA.

---

19. Previous CAPTCHA Must Not Be Reused

This is an important functional requirement.

After a challenge is completed:

- The old CAPTCHA must be discarded
- The old answer options must be discarded
- A new CAPTCHA must be generated
- New answer options must be generated
- The previous challenge must not appear again

This applies to both:

Claim flow

and

No Thanks flow

---

20. Dynamic UI Requirement

The page must **NOT feel static**.

This is one of the highest-priority requirements.

The UI should react naturally to every user interaction.

CAPTCHA Load

When the CAPTCHA page appears:

- Content should enter smoothly
- CAPTCHA should have subtle reveal animation
- Options should appear naturally
- Reward information should animate into place

Hover

When hovering over an answer:

- Option should respond immediately
- Visual hierarchy should change
- Subtle movement/elevation may occur

Selection

When selecting:

- Immediate feedback
- Selected state
- 0.5-second verification animation
- Smooth transition

Checking

The checking page should have continuous but subtle motion.

Result

The result should reveal itself smoothly.

Reward

The Gem reward should have a visually satisfying reveal.

New CAPTCHA

When returning to the CAPTCHA page:

- Old challenge disappears
- New challenge appears
- New options are displayed
- Interface feels refreshed

Avoid simply changing the text inside the same static card.

---

21. Premium Fintech Visual Direction

The visual language should align with VELoop Rewards.

Desired Characteristics

Premium

Modern

Fintech

Trustworthy

Interactive

Rewarding

Clean

Responsive

Polished

---

22. What NOT To Build

Do NOT create:

- Generic CAPTCHA form
- Basic Bootstrap card
- Plain white/black form
- Static dashboard card
- Generic glassmorphism
- Cheap neon design
- Casino-style interface
- Children's game aesthetic
- Crypto exchange interface
- Excessively glowing UI
- Excessive particle effects
- Excessive animations
- Template-looking design

The module should feel like a **real VELoop Rewards product feature**.

---

23. Design Freedom

The intern has complete creative freedom regarding:

- Layout
- Component positioning
- Typography
- Background treatment
- CAPTCHA presentation
- Option design
- Reward presentation
- Animation
- Interaction patterns
- Visual hierarchy
- Button styling

Do not copy an existing UI.

The only things that must remain fixed are the **functional requirements and user flow** defined in this document.

---

24. Reward Should Feel Valuable

The user is performing the activity to earn Gems.

Therefore:

+1 Gem

and

+0.5 Gems

must be visually meaningful.

Do not hide the reward in a small label.

At the same time, avoid making the reward look like gambling.

Think:

Premium fintech reward experience

rather than:

Arcade/casino reward experience

---

25. Responsive Design

The module must work properly on:

- Mobile
- Tablet
- Desktop

Mobile interaction is especially important.

Answer options must be:

- Easy to tap
- Properly spaced
- Large enough
- Comfortable
- Responsive

The layout must not suffer from:

- Horizontal scrolling
- Overlapping components
- Broken animations

- Tiny buttons
- Text clipping
- Layout shifts

---

26. Frontend Technology

Use:

- React
- Modern CSS / CSS Modules / project-approved styling system
- Component-based architecture

Follow the existing VELoop Rewards frontend architecture where applicable.

Keep the implementation:

- Reusable
- Maintainable
- Clean
- Modular

---

27. Suggested Component Structure

One possible structure:

```
CaptchaEarn/
│
├── CaptchaPage
│
├── CaptchaChallenge
│
├── CaptchaOptions
│
├── CaptchaOption
│
├── RewardIndicator
│
├── VerificationState
│
├── CheckingPage
│
├── ResultPage
│   ├── CorrectResult
│   └── WrongResult
│
```

```
├── RewardReveal
│
├── ClaimButton
│
├── NoThanksButton
│
└── MockAdState
```

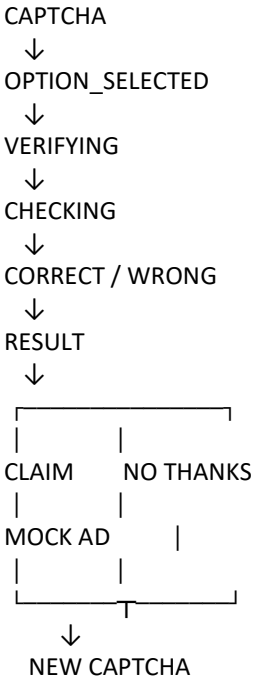
This is only a suggestion.

A better architecture is welcome.

---

28. Frontend State Flow

The module should support states similar to:



---

29. Frontend Scope

Intern MUST build

- CAPTCHA page
- CAPTCHA display
- Four answer options
- Correct answer state
- Wrong answer state

- Option interaction
- 0.5-second verification animation
- Checking page
- Correct result page
- Wrong result page
- +1 Gem reward display
- +0.5 Gem reward display
- Claim button UI
- No Thanks button
- Mock ad state
- CAPTCHA refresh
- Previous CAPTCHA invalidation
- Responsive layout
- Animations
- Micro-interactions

Intern MUST NOT build

- Backend
- Database
- Real CAPTCHA security
- Authentication
- Real reward API
- Wallet API
- Real advertisement integration
- Ad SDK
- Rewarded ad network
- Backend reward validation
- Production reward transaction system

---

30. Development Process

Do NOT immediately start coding.

Follow this process:

Step 1 — Understand the Flow

Understand the complete user journey before implementation.

Step 2 — Create UI Concept

Design the CAPTCHA page and result page first.

Focus on:

- Layout
- Visual hierarchy
- Interaction
- Reward presentation
- Responsive behavior

Step 3 — Review

The UI concept should be reviewed before full implementation.

Step 4 — Implement

Build the approved design in React.

Step 5 — Add Interactions

Implement:

- Selection
- Verification animation
- Checking state
- Result state
- Reward reveal
- No Thanks
- Mock Claim/ad state
- CAPTCHA refresh

Step 6 — Responsive Testing

Test on:

- Mobile
- Tablet
- Desktop

Step 7 — Final Polish

Check:

- Animations
- Spacing
- Typography
- Touch interaction
- Loading states
- Transitions

- Edge cases

---

31. Acceptance Criteria

The module will be considered complete only when:

CAPTCHA

- [ ] Exactly 4 options are displayed
- [ ] Only 1 option is correct
- [ ] 3 options are reasonably similar to the correct answer
- [ ] 1 option is completely different
- [ ] CAPTCHA contains characters and digits

Interaction

- [ ] No Submit button is required
- [ ] Selecting an option immediately starts the verification process
- [ ] Selection animation lasts approximately 0.5 seconds
- [ ] User cannot select another option during verification

Verification

- [ ] Checking state is displayed
- [ ] Checking state is animated
- [ ] Correct/wrong result is displayed afterward

Rewards

- [ ] Correct answer displays +1 Gem
- [ ] Wrong answer displays +0.5 Gems

Result

- [ ] Correct result has Claim and No Thanks
- [ ] Wrong result has Claim and No Thanks

- [ ] Reward is visually prominent

Claim

- [ ] No real ad integration
- [ ] No real reward API
- [ ] Claim has a frontend/mock transition state
- [ ] New CAPTCHA appears after the mock flow

No Thanks

- [ ] Returns to CAPTCHA page
- [ ] Generates a new CAPTCHA
- [ ] Previous CAPTCHA cannot be reused

UI/UX

- [ ] Premium visual appearance
- [ ] Fintech-inspired design
- [ ] Interactive answer options
- [ ] Smooth micro-interactions
- [ ] Dynamic page transitions
- [ ] Reward animation
- [ ] Responsive design
- [ ] No generic/static appearance

---

32. Final Quality Standard

Before submitting, ask:

First Impression

Does this immediately look like a premium VELoop Rewards feature?

Interaction

Does every important interaction visibly respond?

Animation

Are the animations smooth and purposeful rather than excessive?

Reward

Does receiving +1 Gem or +0.5 Gems feel satisfying?

Trust

Does this look like a legitimate fintech/rewards platform?

Originality

Does the design feel specifically created for VELoop Rewards?

Responsiveness

Does it feel polished on both mobile and desktop?

User Experience

Would a real user enjoy completing multiple CAPTCHA challenges?

If the answer is **no**, continue improving the UI before considering the task complete.

Final Objective

The purpose of this task is **not simply to build a CAPTCHA form**.

The purpose is to turn a normally boring activity into a polished VELoop Rewards earning experience.

The ideal experience is:

See CAPTCHA
↓
Understand challenge
↓
Choose answer
↓
Instant interaction
↓
0.5s verification
↓
Checking

↓
Result
↓
Receive Gems
↓
Claim / No Thanks
↓
Fresh CAPTCHA
↓
Continue earning

The user should finish the first challenge and think:

“This feels surprisingly smooth and premium for a CAPTCHA earning feature.”

That is the quality bar for this task.

51. Project Timeline s Submission Deadline

To ensure smooth project execution and timely review:

Project Assigned On:	**25 August 2026**
Project Start Date:	**25 August 2026**
Submission Deadline:	**10 September 2026**
Submission Tine:	**5:30 P.M. (IST)**